

סדנה מתקדמת בתכנות

פרק 5 : מערכים דינאמיים דו-ממדיים

חזרה: מערכים דו-ממדיים סטטיים

- לפעמים נרצה לייצג מטריצות (טבלאות), או מערכים ממימדים גבוהים יותר.
- נוכל להגדיר מערכים כאלה למשל על-ידי:

```
int a[30][20]; // 30 rows, 20 columns
```

- ואז נוכל לגשת לכל תא ע"י שני אינדקסים, למשל:

```
a[5][8]=4; // row 5, column 8
```

```
int a[3][4];
```

	Column 0	Column 1	Column 2	Column 3
Row 0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
Row 1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
Row 2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

Diagram illustrating the structure of a 2D array `a` with 3 rows and 4 columns.

The array is represented as a table with rows and columns. The row indices are 0, 1, and 2. The column indices are 0, 1, 2, and 3. The elements are accessed using the syntax `a[row][column]`.

Labels and arrows pointing to the corresponding parts of the array structure:

- Array name: `a`
- Row subscript: `[row]`
- Column subscript: `[column]`

מערכים דו-ממדיים - אתחול

- ניתן לתת ערכים התחלתיים גם כשמגדירים מערך רב-מימדי, למשל:

1	0
0	1
1	1

```
int my_matrix[3][2]={ {1,0}, {0,1}, {1,1} };
```

1	2	3
4	5	6

```
int arr1[2][3]={ {1,2,3}, {4,5,6} };
```

1	2	0
4	0	0

```
int arr2[2][3]={ {1,2}, {4} };
```

1	2	3
4	5	0

```
int arr3[2][3]={ 1,2,3,4,5 };
```

מערכים דו-ממדיים כפרמטרים של פונקציות

■ אם צריך לעביר מערך דו-ממדי לפונקציה כפרמטר יש לרשום `name[][N]`.

■ N הוא מספר שלם (קבוע) המציין מספר עמודות במערך

כתובת תחילת המערך

מספר שורות

■ דוגמאות:

■ `void print_matrix ( int a[][10], int rows, int cols )`
`cols <= 10`

מספר עמודות

■ `#define N 10`
`double sum_elements ( double b[][N], int m, int n )`
`n <= N`

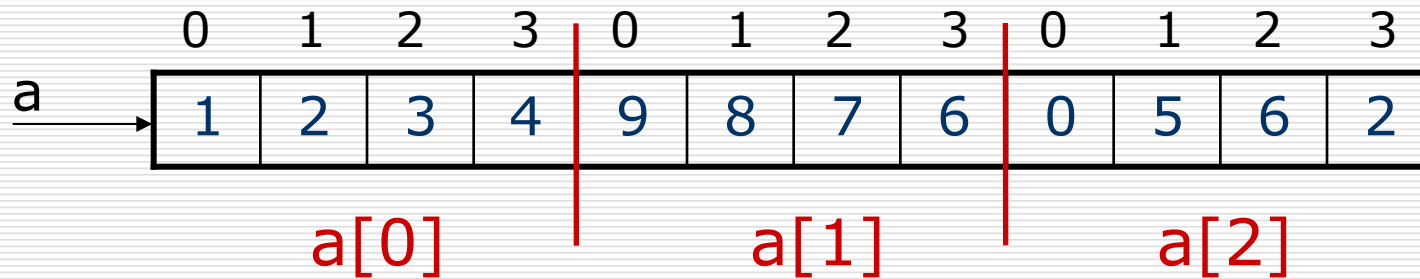
■ למה?

מערכים דו-ממדיים כפרמטרים של פונקציות

```
int a[3][4]={ {1,2,3,4},  
              {9,8,7,6},  
              {0,5,6,2} };
```

	0	1	2	3
0	1	2	3	4
1	9	8	7	6
2	0	5	6	2

לוגית:



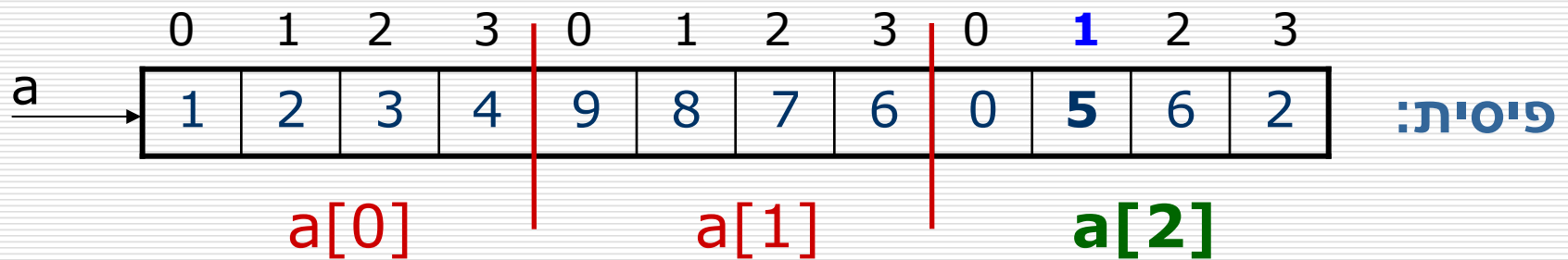
פיסית:

גודל שורה (מספר עמודות) הוא גם מאפיין טיפוס
איברי המערך החד-ממדי ולכן צריך לציין אותו

מערכים דו-ממדיים כפרמטרים של פונקציות

```
int a[3][4]={ {1,2,3,4},  
              {9,8,7,6},  
              {0,5,6,2} };
```

$a[2]$ הוא שם המערך. לפניה לאיבר מס' 1 בתוך מערך צריך לרשום שם המערך ו- $[1]$ מצד ימין. ככה מקבלים $a[2][1]$ שהינו איבר של מערך דו-ממדי.

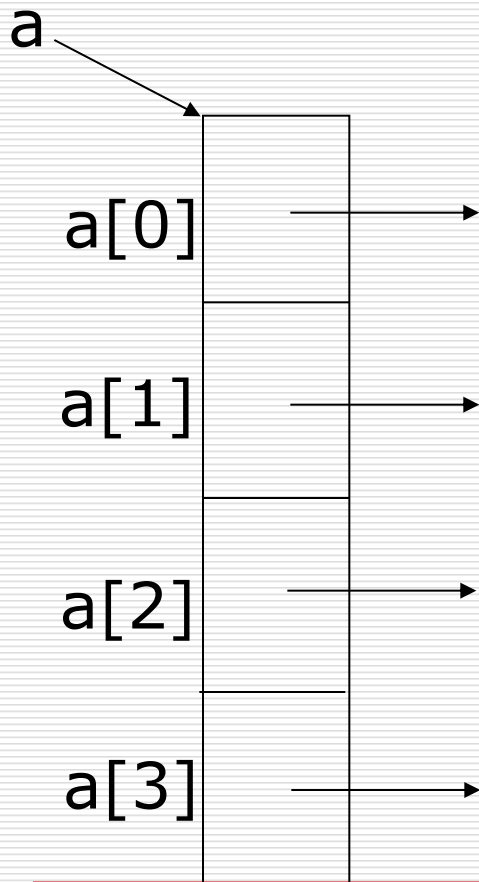


גודל שורה (מספר עמודות) הוא גם מאפיין טיפוס איברי המערך החד-ממדי ולכן צריך לציין אותו

מערך מצביעים

```
int *a[4], i;
```

נקצה מערכים דינאמיים באמצעות כל
מצביעי מערך a

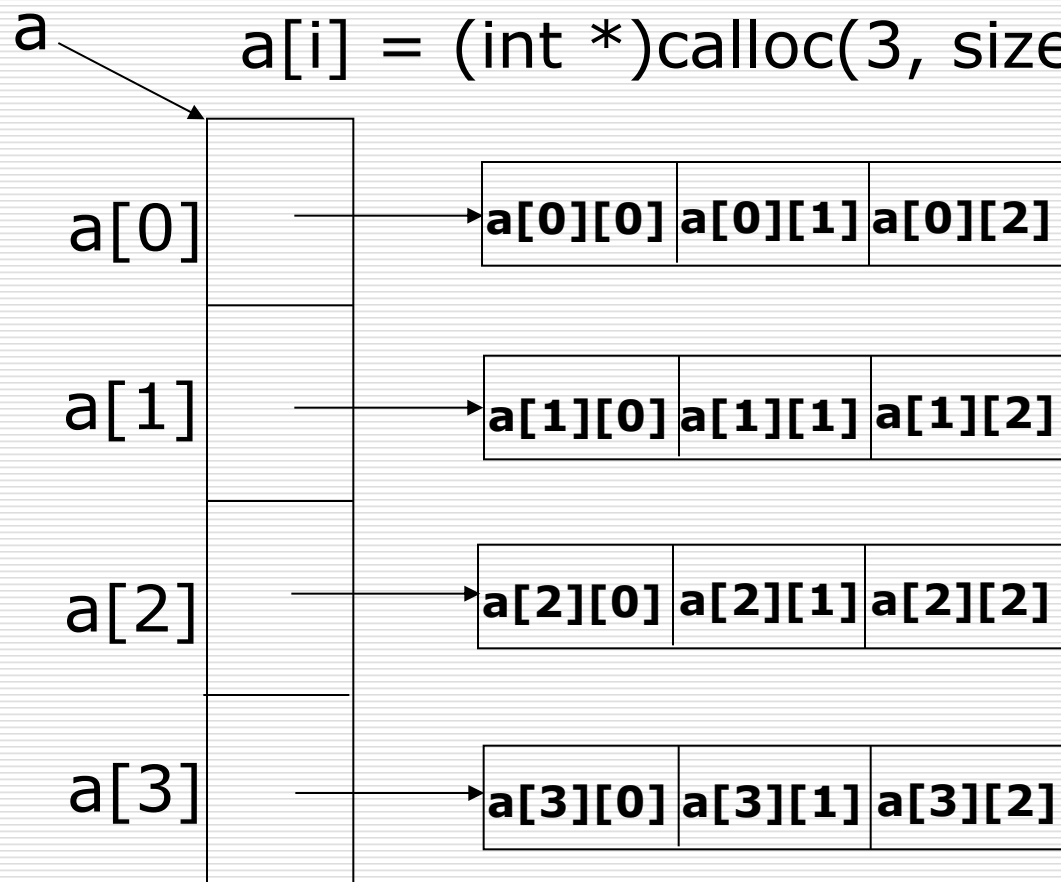


מערך מצביעים כבסיס למערך דו-ממדי

```
int *a[4], i;  
for (i=0; i<4; i++)
```

קיבלנו מערך דו-ממדי

```
    a[i] = (int *)calloc(3, sizeof(int));
```

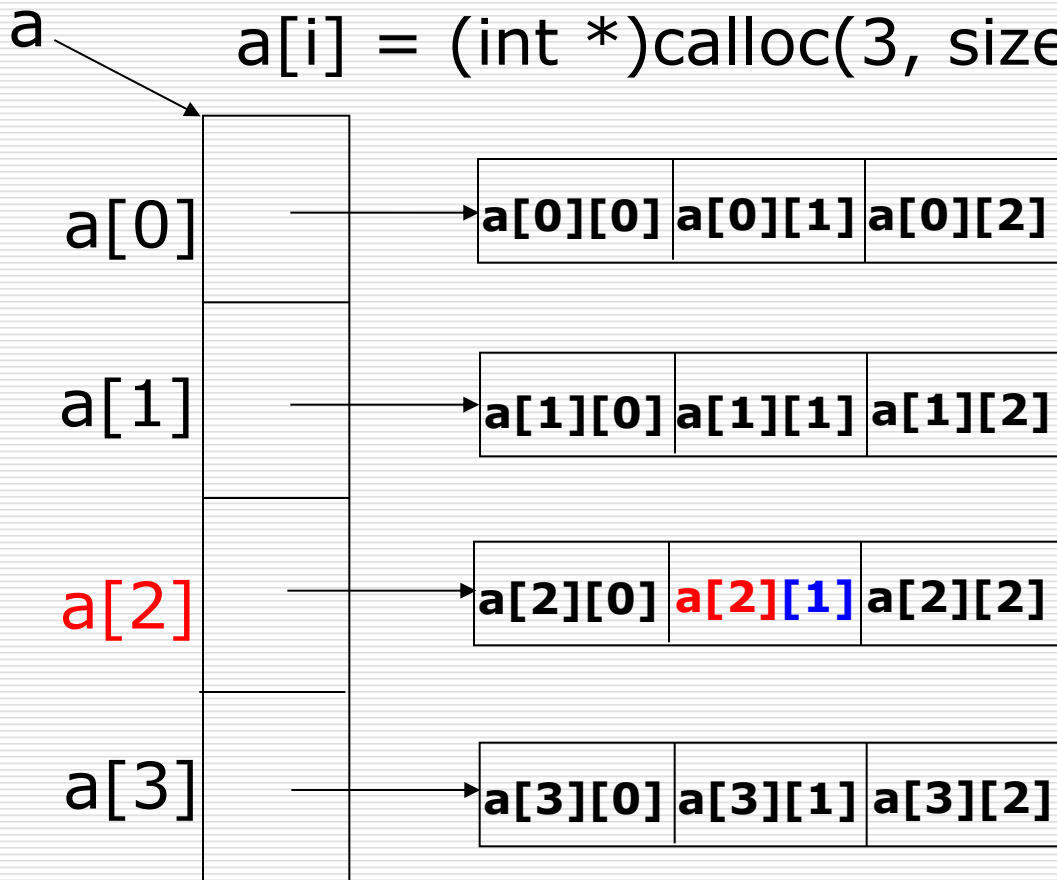


מערך מצביעים כבסיס למערך דו-ממדי

```
int *a[4], i;  
for (i=0; i<4; i++)
```

קיבלנו מערך דו-ממדי

```
    a[i] = (int *)calloc(3, sizeof(int));
```



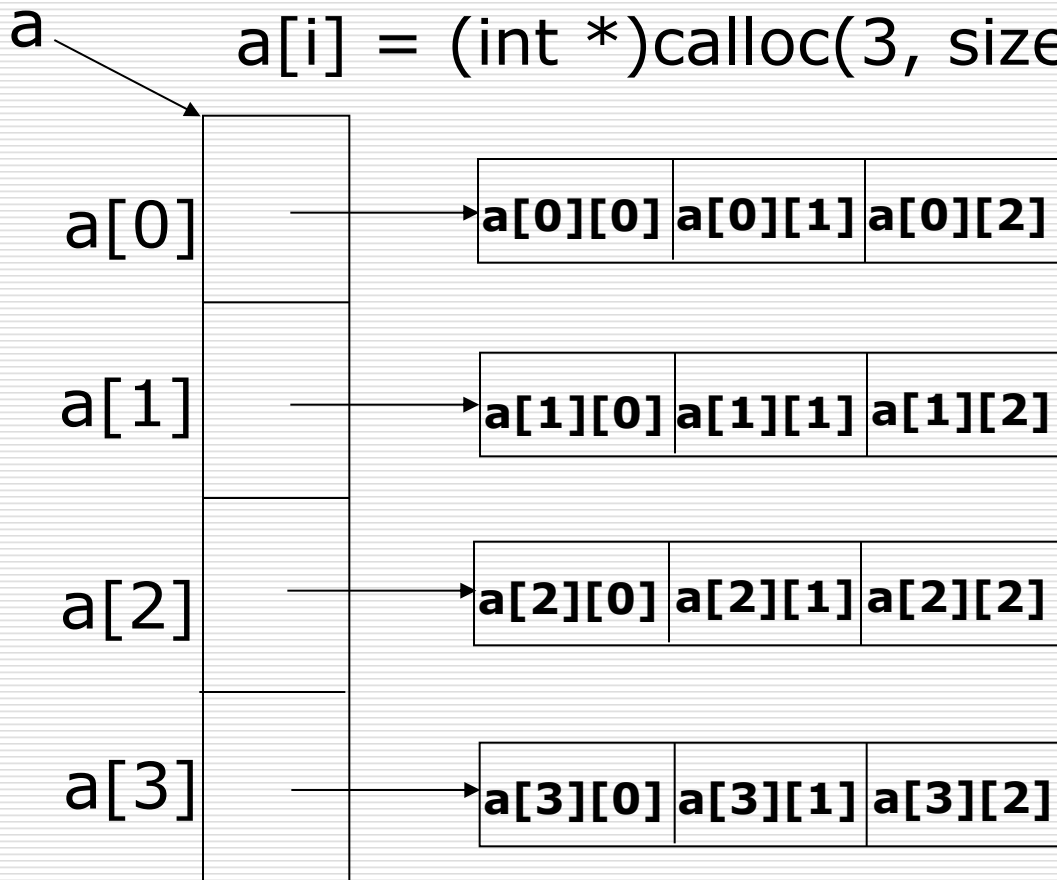
`a[2]` הוא שם המצביע וגם שם המערך הדינאמי.
לפניה לאיבר מס' 1 בתוך מערך צריך לרשום שם המערך ו-**`[1]`** מצד ימין.
ככה מקבלים **`a[2][1]`** שהינו איבר של מערך דו-ממדי.

מערך מצביעים כבסיס למערך דו-ממדי

```
int *a[4], i;
```

```
for (i=0; i<4; i++)
```

```
    a[i] = (int *)calloc(3, sizeof(int));
```



ניתן לפנות לאיבר $a[i][j]$
במערך דו-ממדי גם ככה:

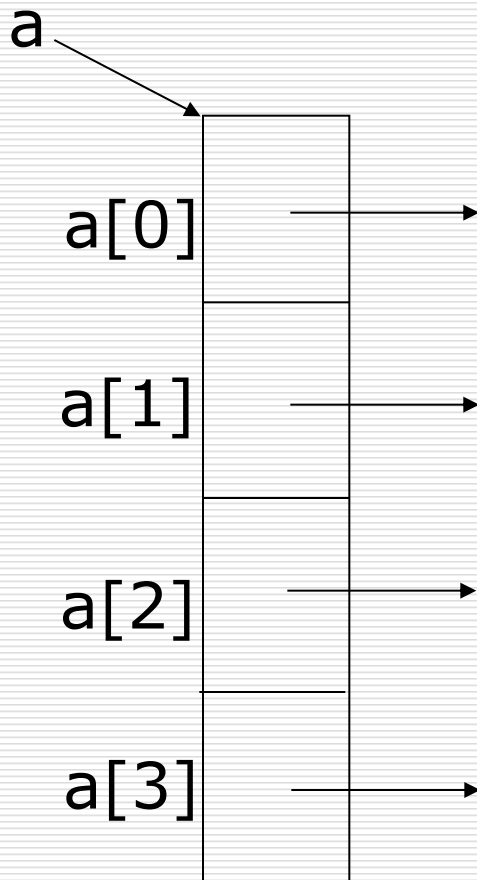
$*(*(a+i)+j)$

מערך מצביעים כבסיס למערך דו-ממדי

```
int *a[4], i, n;  
for (i=0; i<4; i++)  
{  
    scanf ("%d", &n);  
    a[i] = (int *)calloc(n, sizeof(int));  
}
```

- ניתן להקצות כל שורה בדיוק לפי הגודל הנדרש
- גדלי שורות במערך יכולים להיות שונים
- זה מונע בזבוז משאבי זיכרון

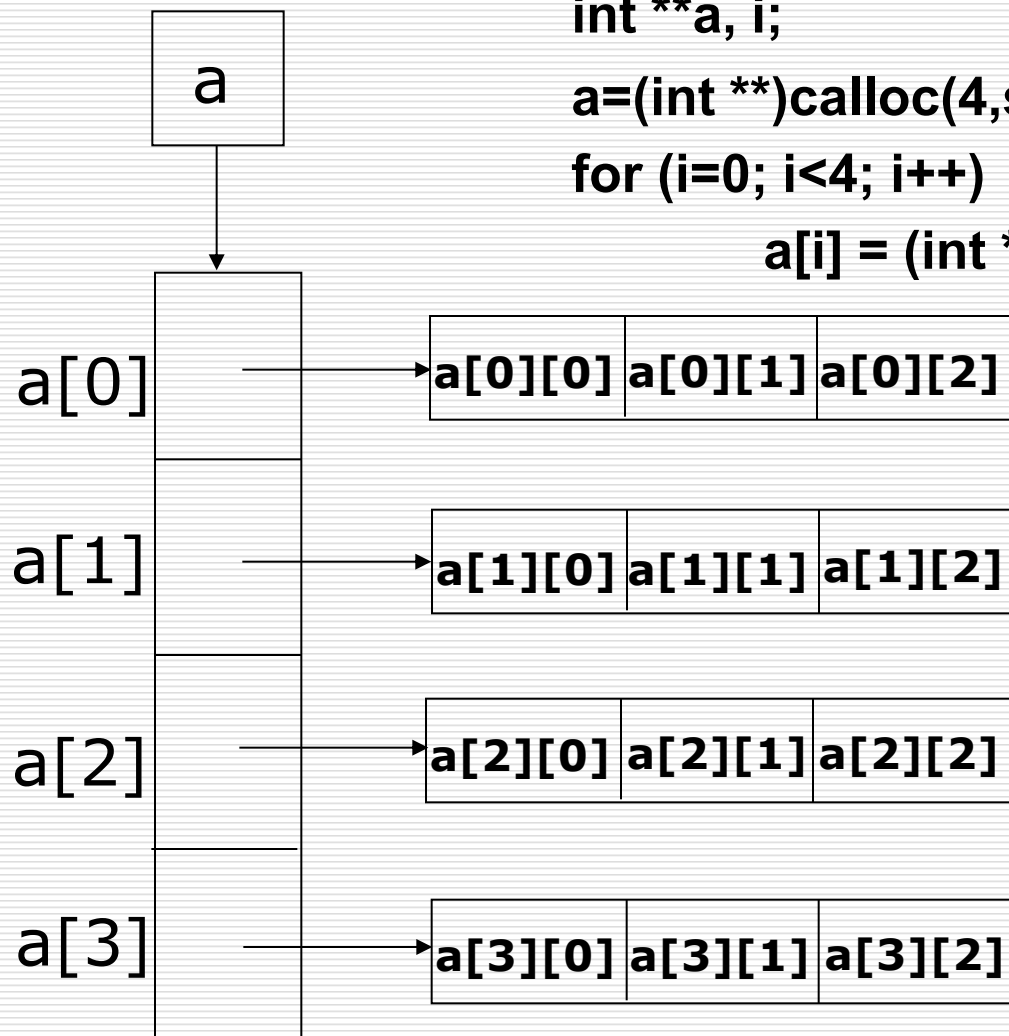
```
int *a[4];
```



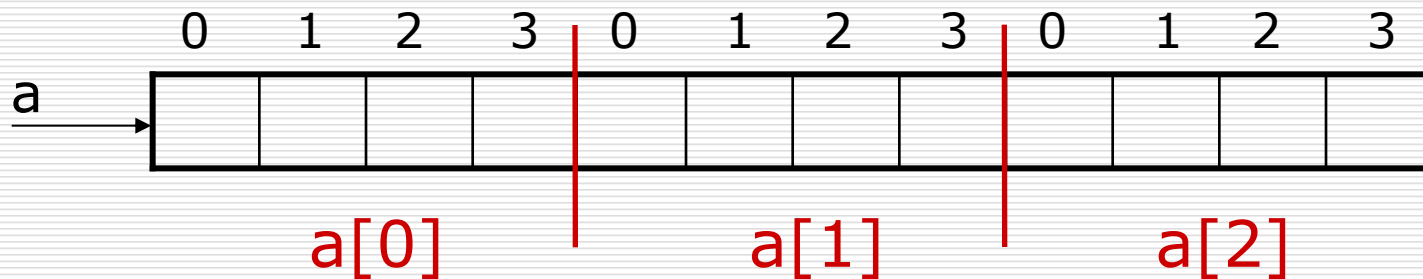
- **a** הוא מערך סטטי של מצביעים וגם מצביע לתחילת המערך
- לכן **a** הוא מצביע למצביע קבוע
- מערך מצביעים גם יכול להיות דינאמי

מערך דינאמי דו-ממדי

```
int **a, i;  
a=(int **)calloc(4,sizeof(int*));  
for (i=0; i<4; i++)  
    a[i] = (int *)calloc(3, sizeof(int));
```



```
int a[3][4], **p;
```



`p = a;` – Warning: `int **` differs in levels of indirection from `int (*)[4]`

הזהרה : אי-התאמת טיפוסים!

העברת מערכים דינאמיים דו-ממדיים לפונקציות

- מערך דינאמי: צריך להעביר מצביע למצביע, מספר שורות, מספר עמודות

דוגמא: `void func_name(int **a, int rows, int cols);`

הפונקציה היא אוניברסאלית גם מבחינת מספר שורות וגם מבחינת מספר עמודות

- מערך סטטי:

דוגמא: `void func_name(int a[][4], int rows, int cols);`

או `void func_name(int (*a)[4], int rows, int cols);`

הפונקציה היא אוניברסאלית רק מבחינת מספר שורות

- למערך סטטי ולמערך דינאמי צריך להשתמש בפונקציות שונות!

```
void print_din_matrix(int **a,int rows,int cols)
{
    int i,j;
    for(i=0;i<rows;i++)
    {
        for(j=0;j<cols;j++)
            printf("%8d",a[i][j]);
        printf("\n");
    }
}
```



```
void free_matrix (int **c, int n)
{
    int i;
    for (i=0; i<n; i++)
        free (c[i]);
    free (c);
}
```

```
int **alloc_matrix (int rows, int cols)
{
    int **c, i;
    c = (int **)calloc(rows, sizeof(int *));
    if (!c)
        return NULL;
    for (i=0; i< rows; i++)
    {
        c[i] = (int *)calloc(cols, sizeof(int));
        if (!c[i])
        {
            free_matrix (c, i);
            return NULL;
        }
    }
    return c;
}
```

/* 1. Memory for new matrix is allocated in main.
Address of new matrix is passed to function. */

```
void add_matrices_1 (int **a, int **b, int n, int m, int **c)
{
    int i, j;
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            c[i][j] = a[i][j] + b[i][j];
}
```

/* 2. Memory for new matrix is allocated in function.
Address of new matrix is returned by function into main. */

```
int **add_matrices_2 (int **a, int **b, int n, int m)
{
    int **c, i, j;
    if ( !(c = alloc_matrix(n,m)) )
        return NULL;
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            c[i][j] = a[i][j] + b[i][j];
    return c;
}
```

/* 3. Memory for new matrix is allocated in function.
Address of new matrix is passed by reference. */

```
void add_matrices_3 (int **a, int **b, int n, int m, int ***pc)
{
    int i, j;
    if ( !(*pc = alloc_matrix(n,m)) )
        return;
    for (i=0; i<n; i++)
        for (j=0; j<m; j++)
            (*pc)[i][j] = a[i][j] + b[i][j];
}
```

שימוש בפונקציות לחיבור מטריצות

```
int **A, **B, **C, **D, **E, Rows, Cols;
```

Determining Rows, Cols

```
A = alloc_matrix(Rows, Cols);
```

```
B = alloc_matrix(Rows, Cols);
```

```
C = alloc_matrix(Rows, Cols);
```

Filling A, B

```
add_matrices_1 (A, B, Rows, Cols, C);
```

```
D=add_matrices_2 (A, B, Rows, Cols);
```

```
add_matrices_3 (A, B, Rows, Cols, &E);
```

העברת מערך דינאמי רב-ממדי מפונקציה

type ***...* func_name (...)
 └──┬──┘
 n

קריאה:

type ***...* point;
 └──┬──┘
 n

point = func_name(...);

void func_name (... , **type** ***...* ptr)
 └──┬──┘
 n+1

קריאה:

type ***...* point;
 └──┬──┘
 n

func_name(..., &point);

מערכים סטטיים, דינאמיים, דינאמיים חלקית

```
#define M 3
```

```
#define N 4
```

```
int a[M][N], // מערך סטטי של M מערכים בעלי N שלמים ז"א מצביע קבוע למערך בעל N שלמים  
    **b, // מצביע למצביע ל-int  
    (*c)[N], // מצביע למערך בעל N שלמים  
    *d[M]; // מערך סטטי של M מצביעים ל-int ז"א מצביע קבוע למצביע ל-int
```

```
// מערך דינאמי
```

```
b=(int **)calloc(M, sizeof(int *));  
for (i=0; i<M; i++)  
    b[i]=(int *)calloc(N, sizeof(int));
```


מערכים סטטיים, דינאמיים, דינאמיים חלקית

```
#define M 3
```

```
#define N 4
```

```
int a[M][N], // מערך סטטי של M מערכים בעלי N שלמים ז"א מצביע קבוע למערך בעל N שלמים
```

```
**b, // מצביע למצביע ל-int
```

```
(*c)[N], // מצביע למערך בעל N שלמים
```

```
*d[M]; // מערך סטטי של M מצביעים ל-int ז"א מצביע קבוע למצביע ל-int
```

```
// מערך דינאמי חלקית
```

```
for (i=0; i<M; i++)
```

```
    d[i]=(int *)calloc(N, sizeof(int));
```

מערכים סטטיים, דינאמיים, דינאמיים חלקית

```
#define M 3
#define N 4
typedef int arrayN[N]; // arrayN מוגדר כטיפוס שלמים בעל N
int a[M][N], // מערך סטטי של M מערכים בעלי N שלמים ז"א מצביע קבוע למערך בעל N שלמים
    **b, // מצביע למצביע ל-int
    (*c)[N], // מצביע למערך בעל N שלמים
    *d[M]; // מערך סטטי של M מצביעים ל-int ז"א מצביע קבוע למצביע ל-int
arrayN *e; // מצביע לטיפוס arrayN ז"א מצביע למערך בעל N שלמים

// מערך דינאמי חלקית
c=(int (*) [N])calloc(M, sizeof(int [N]));
c=(arrayN *)calloc(M, sizeof(arrayN)); // the same
e= (arrayN *)calloc(M, sizeof(arrayN));
```

התאמה בין מצביעים שונים

```
#define M 3
#define N 4
typedef int arrayN[N]; // arrayN מוגדר כטיפוס שלמים בעל N
int a[M][N], // מערך סטטי של M מערכים בעלי N שלמים ז"א מצביע קבוע למערך בעל N שלמים
    **b, // מצביע למצביע ל-int
    (*c)[N], // מצביע למערך בעל N שלמים
    *d[M], // מערך סטטי של M מצביעים ל-int ז"א מצביע קבוע למצביע ל-int
    (*f)[5]; // מצביע למערך בעל 5 שלמים
arrayN *e; // מצביע לטיפוס arrayN ז"א מצביע למערך בעל N שלמים
```

סיבת שגיאה: השמה לקבוע

```
c=a;
```

```
a=c; //error
```

```
b=d;
```

```
d=b; //error
```

התאמה בין מצביעים שונים

```
#define M 3
#define N 4
typedef int arrayN[N]; // arrayN מוגדר כטיפוס שלמים בעל N
int a[M][N], // מערך סטטי של M מערכים בעלי N שלמים ז"א מצביע קבוע למערך בעל N שלמים
    **b, // מצביע למצביע ל-int
    (*c)[N], // מצביע למערך בעל N שלמים
    *d[M], // מערך סטטי של M מצביעים ל-int ז"א מצביע קבוע למצביע ל-int
    (*f)[5]; // מצביע למערך בעל 5 שלמים
arrayN *e; // מצביע לטיפוס arrayN ז"א מצביע למערך בעל N שלמים
```

סיבת אזהרה: אי-התאמת טיפוסים

סיבת שגיאה: השמה לקבוע

```
a=d; //error and warning    a=b; //error and warning
```

```
d=a; //error and warning    b=a; //warning
```

```
c=d; //warning
```

```
d=c; //error and warning
```

ערכים: לאוניד קוגל & מרק קורנבליט

התאמה בין מצביעים שונים

```
#define M 3
#define N 4
typedef int arrayN[N]; // arrayN מוגדר כטיפוס שלמים בעל N
int a[M][N], // מערך סטטי של M מערכים בעלי N שלמים ז"א מצביע קבוע למערך בעל N שלמים
    **b, // מצביע למצביע ל-int
    (*c)[N], // מצביע למערך בעל N שלמים
    *d[M], // מערך סטטי של M מצביעים ל-int ז"א מצביע קבוע למצביע ל-int
    (*f)[5]; // מצביע למערך בעל 5 שלמים
arrayN *e; // מצביע לטיפוס arrayN ז"א מצביע למערך בעל N שלמים
```

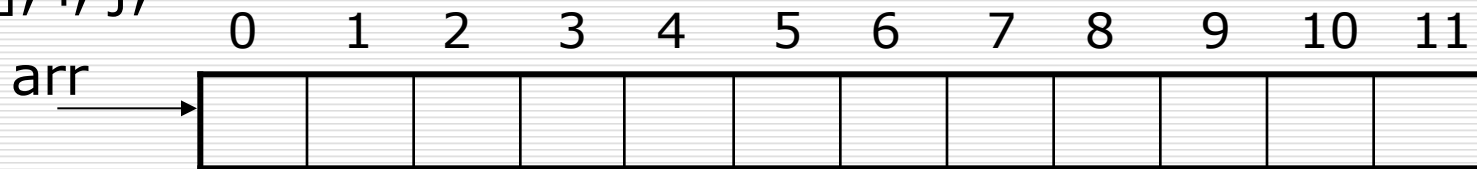
סיבת אזהרה: אי-התאמת טיפוסים

```
c=e;
e=c;
```

```
c=f; //warning
f=c; //warning
```

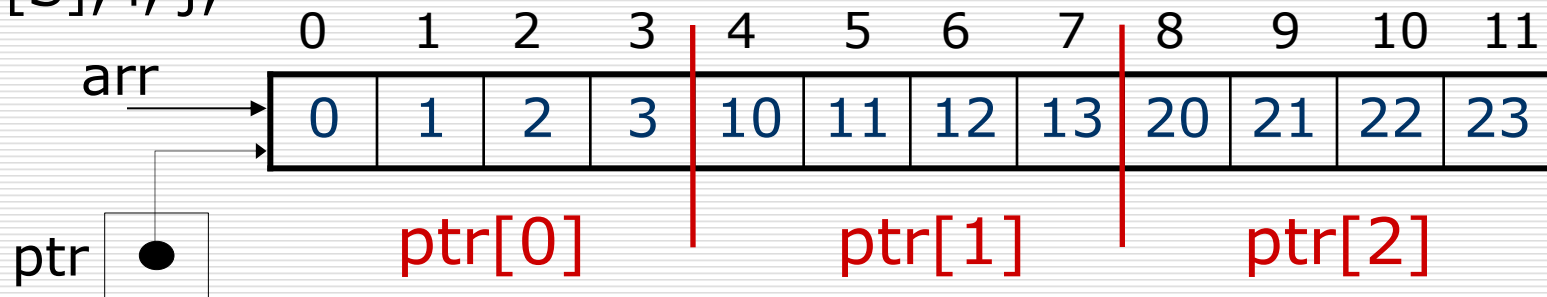
התאמה בין מערכים חד-ממדיים לדו-ממדיים

```
#define M 3
#define N 4
#define S M*N
typedef int arrayN[N]; // מערך בעל N שלמים מוגדר כטיפוס arrayN
int arr[S], i, j;
```



התאמה בין מערכים חד-ממדיים לדו-ממדיים

```
#define M 3
#define N 4
#define S M*N
typedef int arrayN[N]; // מערך בעל N שלמים מוגדר כטיפוס arrayN
int arr[S], i, j;
```



```
arrayN *ptr;
ptr=(arrayN *)arr;
for (i=0; i<M; i++)
    for(j=0; j<N; j++)
        ptr[i][j] = 10*i+j;
```

מבחינת `ptr` זה מערך דו-ממדי.
מעריך `arr` אפשר היה להגדיר גם כדינאמי.